

Module 3 Lab: Version Control and Collaborative Tooling

Field	Value
Module	M3 Git and Tooling
Exercises	Setup (GitHub + push), 1 (Modern Syntax), 2 (Detached HEAD + reflog), 3 (Atomic Commits), 4 (Merge Conflict), 5 (Squash Workflow)
After this session you can show	A GitHub repository with your real TMS code, safe branching with switch/restore, commit recovery with reflog, atomic conventional commits, a resolved merge conflict, and a squashed feature branch

Welcome to the TMS DevOps Environment

In M1 and M2, you built the TMS domain engine in C# and the typed data layer in TypeScript. But enterprise software is never built alone. Your tms-client project currently lives only on your machine if your hard drive fails, everything is gone. If Yared in Addis Ababa and Sarah in London need to work on the same codebase, there is no way to synchronise.

This lab puts your real code under version control. By the end, your tms-client project lives on GitHub, and you can branch, commit, recover lost work, resolve conflicts, and clean up history the exact workflow every professional development team uses daily.

Setup: Put Your TMS Code on GitHub

This is not a practice exercise this is the real setup you will use for the rest of the programme. Every module from here forward works from this repository.

Step 1 Create a GitHub Repository

1. Go to github.com/new
2. Repository name: tms-client
3. Description: Training Management System TypeScript data layer
4. Visibility: **Public** (so your facilitator and teammates can see your work)
5. Do **not** add a README, .gitignore, or licence you will create these locally

Step 2 Initialise Git in Your M2 Project

Open a terminal in your tms-client folder (the project you built in M2):

`git init`

Step 3 Create a .gitignore

Before your first commit, tell Git which files to ignore. Node.js projects generate a `node_modules/` folder containing thousands of dependency files these should never be committed.

Create a file called `.gitignore` in the project root:

```
node_modules/  
dist/  
*.js  
*.js.map  
*.d.ts  
!src/**/*.d.ts
```

Why exclude `.js` files? Your project is TypeScript the `.ts` files are the source code. The `.js` files are compiled output that can be regenerated with `npx tsc`. Committing compiled output pollutes the repository and causes merge conflicts on files nobody wrote by hand.

Step 4 Make Your First Commit

`git add .`
`git status`

What you should see: Your model files (`models/student.model.ts`, `models/course.model.ts`, etc.), `index.ts`, `tsconfig.json`, `package.json`, and `.gitignore` are staged. `node_modules/` is **not** listed the `.gitignore` is working.

`git commit -m "feat: add TMS TypeScript data layer from M2"`

Step 5 Connect to GitHub and Push

Replace YOUR-USERNAME with your actual GitHub username:

```
git remote add origin https://github.com/YOUR-USERNAME/tms-client.git  
git branch -M main  
git push -u origin main
```

Step 6 Verify on GitHub

Open your repository in a browser. You should see:

- `models/` folder with 5 `.model.ts` files
- `index.ts`
- `tsconfig.json`
- `package.json`
- `.gitignore`

- No node_modules/ folder

Troubleshooting:

Problem	Likely cause	Fix
node_modules/ appears on GitHub	.gitignore was added after the first git add	Run <code>git rm -r --cached node_modules</code> then commit again
error: remote origin already exists	You already added a remote	Run <code>git remote set-url origin https://github.com/YOUR- USERNAME/tms-client.git</code>
Authentication failed	GitHub requires a Personal Access Token (PAT) for HTTPS	Generate a PAT at <code>github.com/settings/tokens</code> and use it as your password

Exercise 1: Modern Git Syntax (switch and restore)

The situation: The legacy developers who set up the TMS project learned Git in 2018. They use `git checkout` for everything switching branches, restoring deleted files, creating new branches. This overloaded command is dangerous and causes accidental data overrides.

Modern Git (2.23+) strictly separates these operations: `git switch` for branches, `git restore` for files.

Think about this: If you modify `models/student.model.ts` and then type `git checkout main`, Git might either switch your branch to `main`, or overwrite `student.model.ts` with the version from `main`. Which one will it do? Because `checkout` is ambiguous, many developers have accidentally wiped out hours of work.

The problem is that **the `checkout` command family is ambiguous to humans** because very similar looking commands perform very different actions:

```
git checkout main           # switch branch
git checkout -- student.model.ts  # discard changes
git checkout main -- student.model.ts  # restore file from main
```

A single missing `--` or an incorrect argument can change the meaning dramatically. That's what Git's newer commands were designed to avoid.

Step 1 Create a Feature Branch (the safe way)

Modern: clear intent "I am switching to a new branch"
`git switch -c feature/course-capacity`

The old way was `git checkout -b feature/course-capacity`. Both create and switch. The difference: switch will refuse to run if you have uncommitted changes that would be overwritten. checkout might silently discard them.

Step 2 Restore a File (the safe way)

Intentionally break one of your model files open `models/course.model.ts` and delete a few lines. Save the file.

```
# Modern: clear intent "I am restoring a file, not switching branches"  
git restore models/course.model.ts
```

Open the file again your changes are gone, the file is back to its last committed state. With `git checkout models/course.model.ts`, the same thing happens, but the command looks identical to `git checkout some-branch` which does something completely different.

Step 3 Switch Back

```
git switch main
```

Exercise 2: The Timeline Panic (Detached HEAD and reflog)

Imagine: A junior developer on your team was trying to view an old version of the architecture. They ran `git checkout a1b2c3d` to look at an earlier commit. Git placed them in a “Detached HEAD” state. Ignorant of the warning, they spent 3 hours writing the Certificate generation logic, committed it, and then switched back to main.

Their commit vanished completely from the visible history. The project manager is panicking because 3 hours of work are seemingly gone.

In Git, commits are never “deleted” automatically, even if they are not attached to a branch. They float in the background as orphans until the Git Garbage Collector runs (default: 30 days). The tool that finds these invisible commits is `git reflog` a hidden journal of every HEAD movement.

Step 1 Simulate the Disaster

```
# Create a normal commit so we have some history  
git commit --allow-empty -m "feat: adding basic student logic"
```

```
# Detach HEAD (go back one commit without a branch)  
git switch --detach HEAD~1
```

```
# Now commit work while detached this is the "lost" work  
git commit --allow-empty -m "feat: certificate generation logic"
```

Switch back to main the commit vanishes from git log

```
git switch main
```

Check your history:

```
git log --oneline
```

The “certificate generation logic” commit is **gone** from the visible history.

Step 2 Recover with reflog

View the hidden journal of every HEAD movement

```
git reflog
```

Find the line that says commit: feat: certificate generation logic. Copy the commit hash (e.g., e7d4f9a).

Rescue it by creating a new branch pointing at the orphaned commit

```
git branch rescue-certificate YOUR-COMMIT-HASH
```

Verify it exists

```
git switch rescue-certificate
```

```
git log --oneline -3
```

You just recovered work that appeared permanently lost. This is the most common “emergency” question in Git interviews: *“I accidentally lost a commit. How do I get it back?”*

Clean up switch back to main

```
git switch main
```

Exercise 3: Atomic Commit Literacy

The situation: Your teammate submits a Pull Request (PR) to the TMS repository. It contains one massive commit: Fix typos, add the Enrollment Capacity System, and update student routing.

The lead architect rejects the PR immediately. Why?

Monolithic commits are hard to review and dangerous to revert. If the Enrollment System crashes production, reverting the commit also reverts the typo fixes and the routing updates collateral damage.

Atomic commits follow a simple rule: one logical change = one commit. Each commit can be reviewed, reverted, or cherry-picked independently.

You will now create a clean feature branch with atomic commits using the [Conventional Commits](#) format.

From this point, you write the commands yourself. The exact syntax is no longer provided.

INSTRUCTIONS: Create a clean atomic history

*# **TODO** 1: Switch to your 'main' branch*

*# **TODO** 2: Create a new branch named 'feature/enrollment-capacity'*

*# **TODO** 3: Create an empty file named 'EnrollmentValidator.ts' inside models/*

*# **TODO** 4: Stage the file*

*# **TODO** 5: Commit using Conventional Commits format:*

feat(enrollment): add max capacity validator

Stuck? You used `git switch -c` in Exercise 1. You used `git add` and `git commit -m` in the Setup. The commands are the same only the branch name, file name, and commit message change.

Exercise 4: The Merge Conflict (Collaborative Integration)

The situation: You and another developer both modified the same line in the TMS assessment logic. When you try to merge their branch into yours, Git stops and demands you resolve the conflict manually.

This is the single most common friction point in team development. Every professional developer must be able to read conflict markers, make a decision, and complete the merge.

Step 1 Set Up the Conflict

Start from main

`git switch main`

Create a baseline file

`echo "export const PASS_THRESHOLD = 60;" > assessment-config.ts`

`git add assessment-config.ts`

`git commit -m "chore: setup baseline assessment rules"`

Simulate a colleague's change on a separate branch

`git switch -c feature/strict-assessments`

`echo "export const PASS_THRESHOLD = 75;" > assessment-config.ts`

`git commit -am "fix(assessment): increase threshold for honors certification"`

Your parallel change on main

`git switch main`

`echo "export const PASS_THRESHOLD = 50;" > assessment-config.ts`

`git commit -am "fix(assessment): lower threshold for inclusive grading"`

The collision

`git merge feature/strict-assessments`

Git prints: Automatic merge failed; fix conflicts and then commit the result.

Step 2 Resolve the Conflict

Open `assessment-config.ts` in your editor. You will see conflict markers:

```
<<<<<<< HEAD
export const PASS_THRESHOLD = 50;
=====
export const PASS_THRESHOLD = 75;
>>>>>>> feature/strict-assessments
```

Make an engineering decision:

- Keep 50 (your change)
- Keep 75 (their change)
- Choose a different value entirely (e.g., 65 as a compromise)

Delete all conflict markers (`<<<<<<<`, `=====`, `>>>>>>>`) so only valid TypeScript remains.

TODO: Stage the resolved file

TODO: Complete the merge with git commit (use the default merge message)

Step 3 Verify

`git log --oneline --graph -5`

You should see a merge commit with two parent branches joining together.

Exercise 5: The History Cleanup (Safe Squash Workflow)

The situation: Your local commit history on a feature branch is messy:

- chore: started certificate logic
- fix: forgot a semi-colon in PDF generator
- oops: typo in student name
- feat: finish PDF certificate logic

If you push this to the shared repository, the architect will reject it for “polluting the history.” You need to **squash** these four messy commits into a single, professional commit.

Rebase safety rule: Never rebase a branch that has already been pushed to a shared repository. Rebase only local branches before the first push.

Step 1 Simulate the Mess

```
git commit --allow-empty -m "chore: started certificate logic"
git commit --allow-empty -m "fix: forgot semicolon in PDF generator"
git commit --allow-empty -m "oops: typo in student name"
git commit --allow-empty -m "feat: finish PDF certificate logic"
```

Step 2 Squash with reset --soft

Create a clean branch for the squash

```
git switch -c cleanup/certificate-generator
```

Reset the last 4 commits but keep all changes staged

```
git reset --soft HEAD~4
```

Recommit as one clean commit

```
git commit -m "feat(certificate): implement PDF certificate generation"
```

Why reset --soft instead of interactive rebase? Interactive rebase opens a terminal text editor (Vim or Nano) which can trap you if you have not configured it. The reset --soft technique achieves the same result without that risk.

Step 3 Verify

```
git log --oneline -5
```

You should see one clean feat(certificate) commit where four messy commits used to be.

Switch back to main

```
git switch main
```

Module 3 What You Can Now Show

- ☐ Your tms-client repository is live on GitHub with the M2 code, and node_modules/ is excluded by .gitignore
- ☐ git reflog shows the rescued "certificate generation logic" commit, and it is attached to the rescue-certificate branch
- ☐ You have at least one commit using the feat(...) Conventional Commits format that you wrote yourself
- ☐ git log --graph --oneline shows a merge commit node where you resolved the assessment threshold conflict
- ☐ The cleanup/certificate-generator branch has a single squashed commit replacing four messy ones

You are ready to collaborate on the TMS repository. Continue to Module 4.